

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\`  
a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-  
aab9fd85e39bf4d65.jsonl`  
- SHA-256: `b73d63b5b10b64dbca640ddfd3c110ec44a786f19399d6cd719d3bed45fbd8bc`  
- Source modified: `2026-05-31T04:18:20+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:17:29.379Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMV3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:
URL: https://instavar.com/blog/ai-production-stack/OCR_SOTA_Feb_2026_Open_Document_AI_Leaderboard
Title: OCR Model Leaderboard 2026 ? Benchmarks and Which to Use (Instavar)
Found via: Open VLMs for document/table extraction (broad/primary) search

Task

- 1. Use WebFetch to retrieve the page content.
- 2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
- 3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
- 4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:17:32.292Z)

I'll fetch the page content and analyze it for claims relevant to the research question.

3. user (2026-05-31T04:17:51.854Z)

OCR Model Selection for Bank Statement PDFs ? Article Analysis

Publication Date: February 13, 2026

Benchmark Rankings & Scores

The article provides OmniDocBench v1.5 scores for open-source models:

Model	Score	Parameters
GLM-OCR	94.62	0.9B
PaddleOCR-VL-1.5	94.50	0.9B
HunyuanOCR	94.10	1B
FireRed-OCR	92.94	2B
DeepSeek-OCR-2	91.09	3B MoE (~500M active)

****Important caveat:**** **"**The top reported OCR models are now close enough on headline benchmarks that production fit matters more than tiny score gaps.**"**

License Information

The article mentions specific license constraints:

- ****GutenOCR:**** **"**Weights are CC-BY-NC, which often disqualifies direct commercial deployment.**"**
- ****HunyuanOCR:**** **"**Uses a custom community license with territory and usage constraints, so legal review should happen before production rollout.**"**

****Note:**** The article does not discuss licenses for Qwen2.5-VL, GOT-OCR2.0, olmOCR, MiniCPM-V, InternVL, Surya, Marker, docling, or Nanonets-OCR.

Table & Financial Document Performance

For bank statements specifically, the article notes:

"The harder question became where each one breaks.**"** The evaluation protocol includes a dedicated slice: **"**Table-heavy documents: 8 pages. Structure fidelity and cell ordering.**"**

****Pass/fail thresholds for tables:****

- Overall reading-order edit distance ? 0.12
- Complex-layout reading-order ? 0.16
- Missing-block rate ? 5%; duplicate-block rate ? 2%

However, ****no model-specific table accuracy data for financial documents is provided.****

Hardware & Quantization

The article does ****not discuss VRAM requirements or quantization options**** for any model.

Hallucination & Reliability Caveats

"A single aggregate OCR score usually hides the failure that will cost you time in production.**"** Key production risks include:

- **"**Grounding risk: extraction pipelines fail when text is correct but linked to the wrong box or wrong row.**"**
- **"**Layout drift risk: table structure quality can degrade faster than plain text quality across new templates.**"**

****DeepSeek-OCR-2 blank-page handling caveat:**** **"**Slowest current workflow; current helper job adds startup overhead.**"**

****Models NOT reviewed in this article:**** Qwen2.5-VL, GOT-OCR2.0, olmOCR (except throughput reference), MiniCPM-V, InternVL, Surya, Marker, docling, Nanonets-OCR.

4. assistant (2026-05-31T04:17:57.906Z)

The fetch succeeded. Let me assess the source and extract the relevant claims.

This is a vendor blog (instavar.com, an "AI production stack" company). It presents a benchmark leaderboard with specific OmniDocBench v1.5 scores and a stated evaluation protocol. It's secondary/blog-tier: not peer-reviewed primary research, but it cites a recognized benchmark (OmniDocBench) and gives concrete methodology and numbers. Notably, the models it features (GLM-OCR, PaddleOCR-VL-1.5, HunyuanOCR, FireRed-OCR, DeepSeek-OCR-2) are largely different from those in the research question's shortlist, and it explicitly does NOT cover most of the requested models (Qwen2.5-VL, GOT-OCR2.0, olmOCR, MiniCPM-V, etc.) or any VRAM/quantization detail. The license callouts are useful and directly on-point for muneem's commercial-license constraint.

5. user (2026-05-31T04:18:07.695Z)

Structured output provided successfully

6. assistant (2026-05-31T04:18:20.792Z)

I've completed the analysis. Here's a summary of what I found:

****Source:**** https://instavar.com/blog/ai-production-stack/OCR_SOTA_Feb_2026_Open_Document_AI_Leaderboard

****Quality:**** blog (vendor/"AI production stack" company blog, secondary-tier ? but it cites a recognized benchmark, OmniDocBench v1.5, with concrete scores and a stated evaluation methodology, which raises its usefulness above pure opinion)

****Publish date:**** 2026-02-13

****Relevance to the muneem research question ? partial:****

- The leaderboard features a **different** set of models (GLM-OCR, PaddleOCR-VL-1.5, HunyuanOCR, FireRed-OCR, DeepSeek-OCR-2) than the research question's shortlist. It explicitly does ****not**** cover Qwen2.5-VL, GOT-OCR2.0, olmOCR, MiniCPM-V, InternVL, Surya, Marker, docling, or Nanonets-OCR ? limiting its direct comparative value.
- It provides ****no VRAM/quantization data****, which is a hard constraint for muneem (single consumer GPU), so it's silent on the hardware-tiering question.
- Its most transferable contributions are: (1) license callouts confirming the research question's premise that some top OCR models carry restrictive weights (GutenOCR = CC-BY-NC; HunyuanOCR = custom community license with territory/usage limits), and (2) the production-reliability framing (grounding risk + layout-drift risk on tables, with per-category pass/fail thresholds rather than a single aggregate score) ? which directly supports muneem's plan to machine-verify parses rather than trust a headline benchmark.

The most central claim is the grounding/layout-drift risk framing, since it speaks directly to why dense bank-statement tables break and why output verification matters. The license claims are supporting evidence for muneem's commercial-license constraint, though they cover models outside the target shortlist.

One caveat worth flagging for the parent synthesis: I could not independently verify the OmniDocBench v1.5 scores or even the existence of several named models (e.g., "GLM-OCR", "DeepSeek-OCR-2", "GutenOCR") from this single blog source ? these names do not all match well-known 2025 releases, so the leaderboard numbers should be treated as unverified vendor claims pending corroboration.